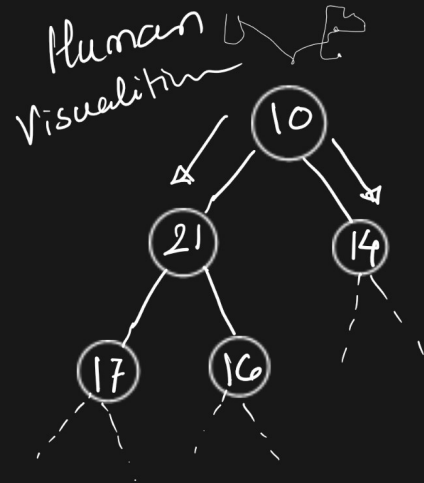
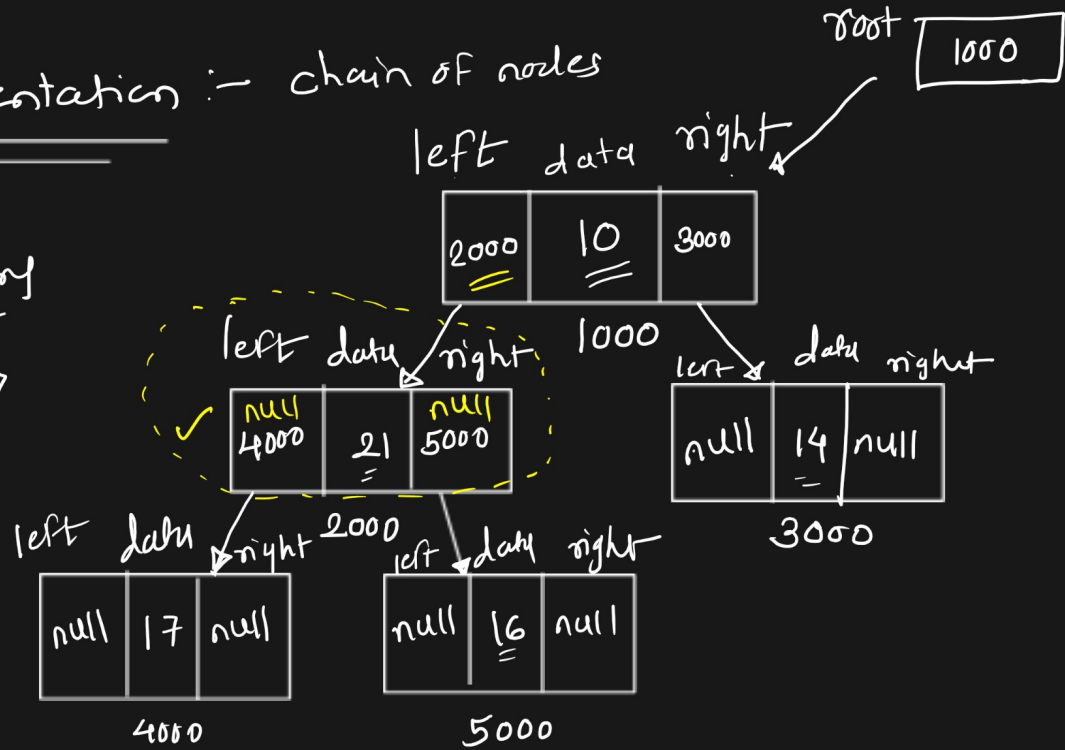


Day No.24 :- Tree Non Linear Data Structure

B) Linked List Representation :- chain of nodes



memory



class TNode

```

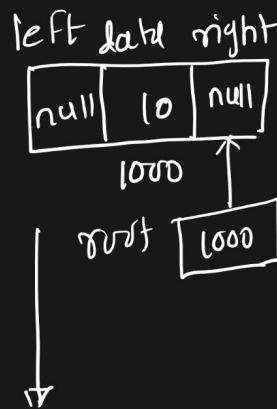
{
    int data;
    TNode left;
    TNode right;

    public TNode(int data)
    {
        this.data = data;
        left = right = null;
    }
}
    
```

public class Main

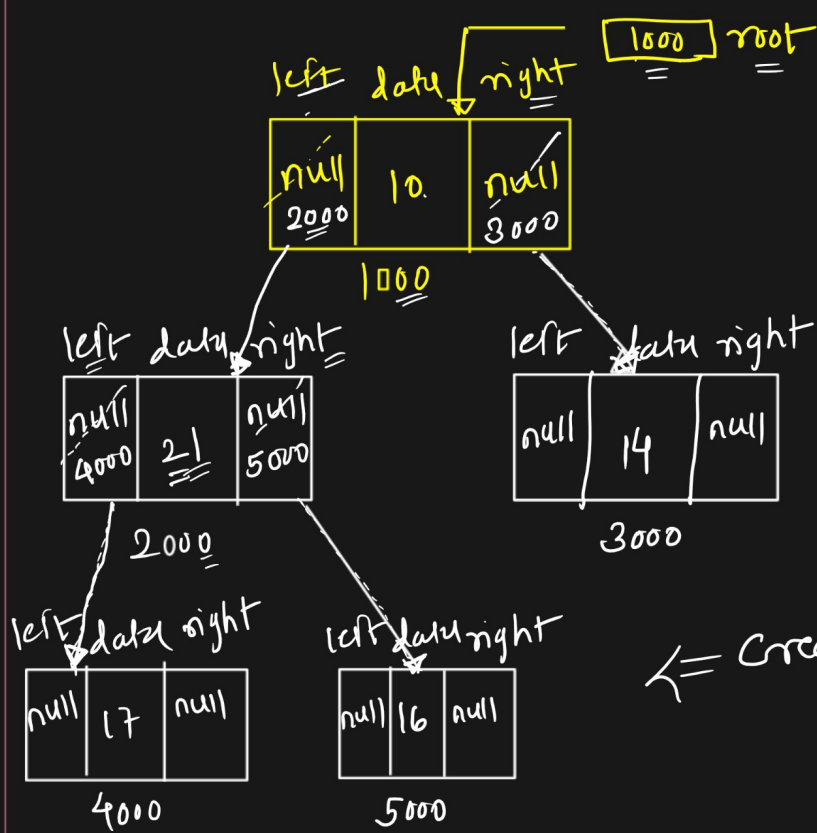
```

{
    p. s. v. m (String args[])
    {
        TNode root = new TNode(10);
        = new TNode(21);
    }
}
    
```



```

TNode root = new TNode(10);
= new TNode(21);
    
```



public class Main

```

{
    p.s.v.m (String anys[])
    {
        TNode root = new TNode(10);
    }
}
  
```

root.left = new TNode(21);

root.right = new TNode(14);

root.left.left = new TNode(17);

root.left.right = new TNode(16);

← create

How we can display the tree elements? ⇒ Traversing inside the tree

For Linear Data Structure How to traverse?

A) Array:- By using indexing

0	1	2	3	4	5

 ⇒ ^{By} index we traverse

B) Stack & Queue
Stack

0

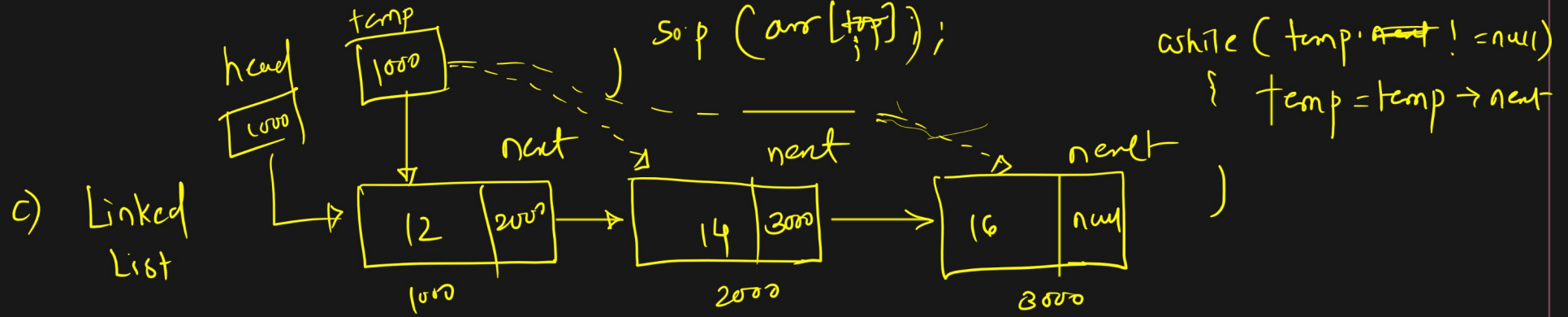
 ← top

By using top variable
for(int i=0; i<top; i++)
{
 Front
 Rear
}

Queue
↓
Front
Rear

0

 ← rear

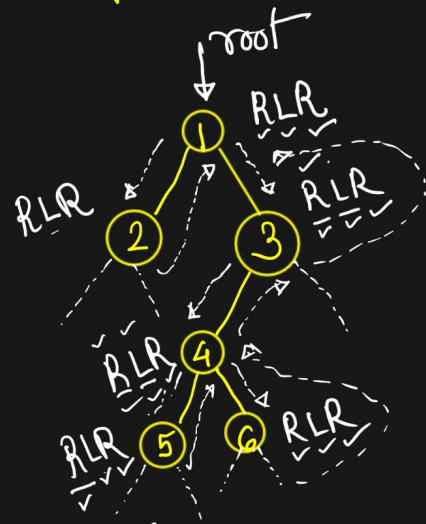


* Tree Traversing :-

Whenever we have to traverse inside the tree, then there are multiple algorithms.

- 1) preorder
- 2) inorder
- 3) postorder
- 4) levelorder

Tree Traversal Algorithm :- Rules of algorithm applicable to each node.



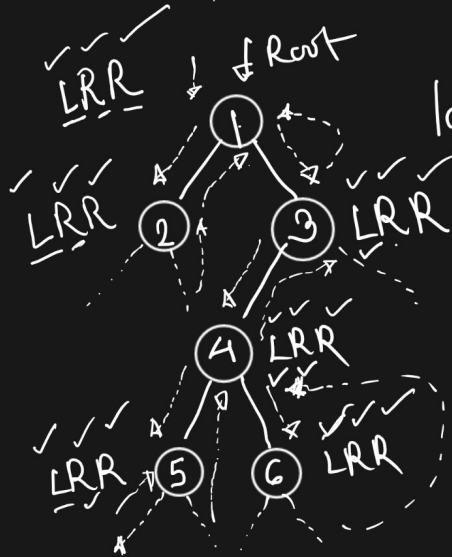
Principle

Result

1) Preorder : $\rightarrow$ Root, Left, Right $\Rightarrow$ 1, 2, 3, 4, 5, 6

2) Inorder : $\rightarrow$ Left, Root, Right $\Rightarrow$ 2, 1, 5, 4, 6, 3

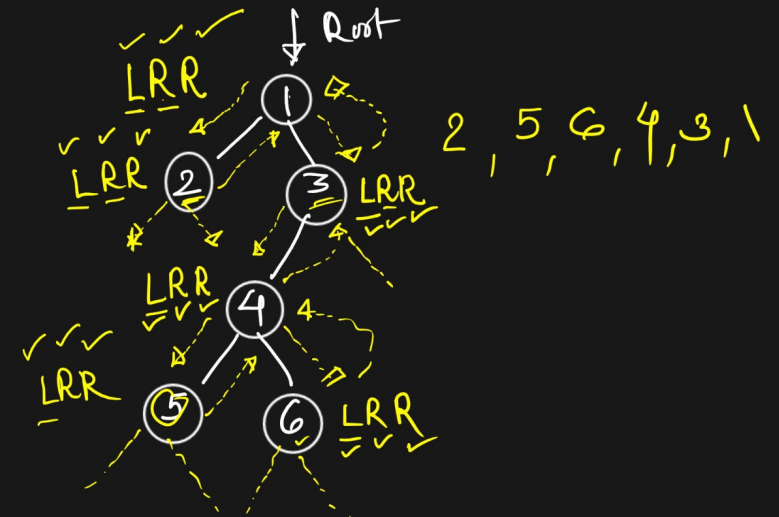
3) Postorder : $\rightarrow$ Left, Right, Root $\Rightarrow$ 2, 5, 6, 4, 3, 1



Inorder :- Left $\rightarrow$ Root $\rightarrow$ Right

2, 1, 5, 4, 6, 3,

Post Order :- Left, Right, Root

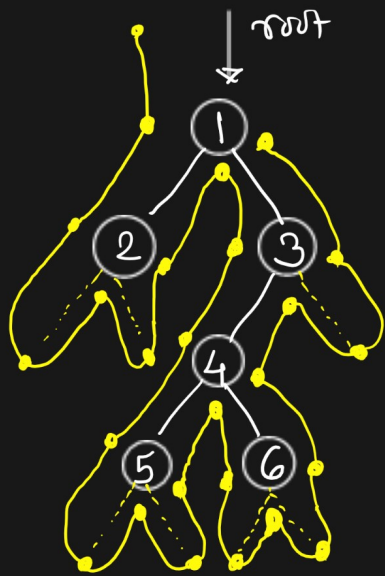


Shortcut way

Rules :- whenever we visited to node as per the multiple time then

rules are

- ✓ 1st ^{time} when we visit any node $\Rightarrow$ preorder
- ✓ 2nd time same node we visit $\Rightarrow$ inorder
- ✓ 3rd time same node we visit $\Rightarrow$ postorder



Tree Traversal Technique

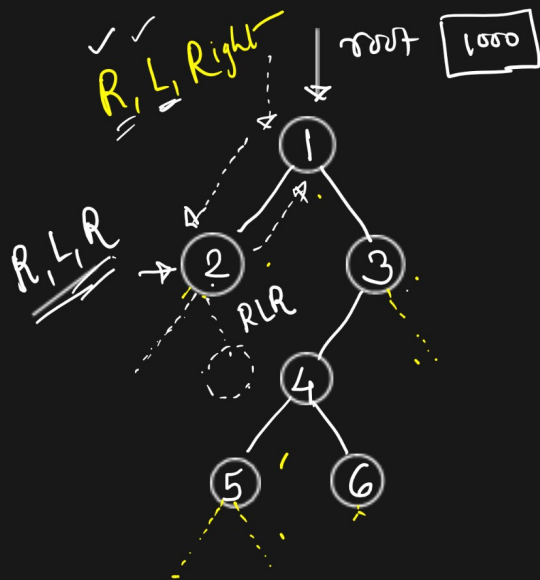
Previous Result

RLR $\Rightarrow$ Preorder $\Rightarrow 1, 2, 3, 4, 5, 6 \Rightarrow \underline{1, 2, 3, 4, 5, 6}$

LRL $\Rightarrow$ Inorder $\Rightarrow 2, 1, 5, 4, 6, 3 \Rightarrow \underline{2, 1, 5, 4, 6, 3}$

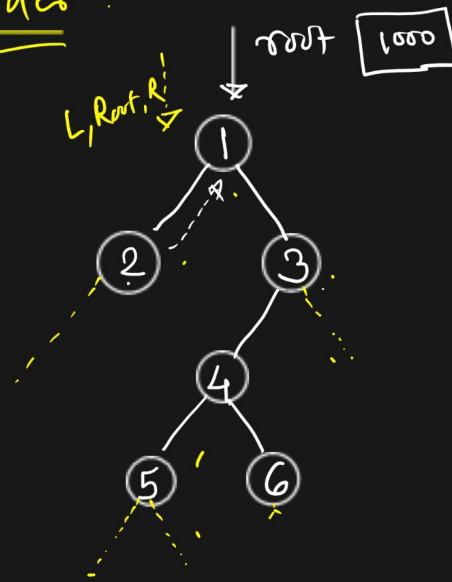
LRRL $\Rightarrow$ Postorder $\Rightarrow 2, 5, 6, 4, 3, 1 \Rightarrow \underline{2, 5, 6, 4, 3, 1}$

* Preorder Traversal (Root, Left, Right) :-



```
void preOrder (TNode root)
{
    if (root != null)
    {
        s.o.p (root.data + " -> ");
        preOrder (root.left);
        preOrder (root.right);
    }
}
```

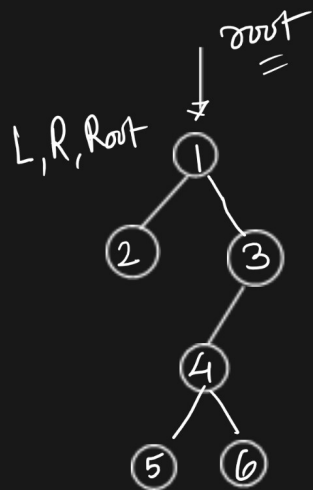

Inorder :- L, Root, R



```

void inOrder(TNode root)
{
    if (root != null)
    {
        inorder(root.left);
        s.o.p. (root.data + " → ");
        inorder(root.right);
    }
}
  
```

3) Post Order Traversing $\Rightarrow$ Left, Right, Root



```

void postOrder(TNode root)
{
    if (root != null)
    {
        postOrder(root.left);
        postOrder(root.right);
        s.o.p. (root.data);
    }
}
  
```